Topic 1 — Network Scanning: Nmap & Zenmap

Aim

1. Perform SYN, FIN, NULL, XMAS and ACK scans and interpret the differences.
2. Perform port scanning and basic firewall detection.
3. Perform service fingerprinting and SNMP/NetBIOS enumeration.

Tools — Nmap (and Zenmap, its GUI front-end), an isolated VirtualBox host-only network, a permitted lab target VM (e.g. Metasploitable2).

Step-by-step (CLI / what Zenmap runs under the hood)

1. Check your own network config first:

```
ipconfig /all
```

2. Confirm the target is reachable:

```
ping 192.168.56.102
```

3. Host discovery / ping sweep (no ports):

```
nmap -sn 192.168.1.0/24
```

4. Default scan (top 1000 ports) then a targeted scan:

```
nmap 192.168.56.102

nmap -p 22,80,443 192.168.56.102
```

5. Service/version detection, save output:

```
nmap -sV 192.168.56.102

nmap -sC -sV 192.168.56.102 -oN scan_results.txt
```

6. The five scan types the exam usually asks you to compare, run on ports 1–100:

```
nmap -sS -p 1-100 192.168.56.102   # SYN — half-open, fast

nmap -sF -p 1-100 192.168.56.102   # FIN — relies on RST/no-response

nmap -sN -p 1-100 192.168.56.102   # NULL — no flags set

nmap -sX -p 1-100 192.168.56.102   # XMAS — FIN+PSH+URG set

nmap -sA -p 1-100 192.168.56.102   # ACK — maps firewall rules, not port state
```

7. SNMP / NetBIOS enumeration (lab only):

nmap -sU -p 161 192.168.56.102

nmap --script nbstat.nse -p 137 192.168.56.102

Likely viva Qs: Why is SYN called a 'half-open' scan? What does ACK scanning actually tell you (vs SYN/FIN/NULL/XMAS)? What does -sV do?

Topic 2 — Network Commands: ping, ifconfig, netstat, traceroute

Aim — Study core diagnostic commands used to check host reachability, interface configuration, active connections and the path packets take to a destination.

Tools — Just a terminal (Windows: cmd/PowerShell · Linux/Mac: bash). No lab VM strictly needed, but demo against your lab target for consistency with other practicals.

Step-by-step

1. **ping** — checks reachability and round-trip latency:

ping 192.168.56.102 # Windows (4 packets, then stops)

ping -c 4 192.168.56.102 # Linux/Mac equivalent

Output shows bytes sent, TTL, and round-trip time; 0% packet loss confirms the host is up.

2. **ifconfig / ipconfig** — shows interface configuration:

ipconfig /all # Windows — IP, subnet, gateway, MAC per adapter

ifconfig -a # Linux/Mac equivalent

Use this to note your IP address, subnet mask, default gateway and MAC address before any scan — establishes your baseline.

3. **netstat** — lists active connections and listening ports:

netstat -ano # Windows: all connections + listening ports + PID

netstat -tulnp # Linux: TCP/UDP listening ports + program name

Columns to point out in viva: Local Address, Foreign Address, State (LISTENING/ESTABLISHED), and PID — useful for spotting an unexpected open port or an unknown process holding a connection.

4. **traceroute / tracert** — shows the hop-by-hop path to a destination:

tracert 8.8.8.8 # Windows

traceroute 8.8.8.8 # Linux/Mac

Each line is one router hop with its round-trip time (×3 probes); * means no reply from that hop (often a firewall). Useful for explaining how TTL expiry is used to map a route, hop by hop.

Likely viva Qs: What does TTL mean and how does traceroute use it? What's the difference between ESTABLISHED and LISTENING in netstat output? Why might a ping succeed but a port scan show the port filtered?

Topic 3 — Reconnaissance and WHOIS

Aim

1. Perform passive and active footprinting of a target domain using only public information.
2. Use WHOIS/ARIN lookups and search-engine recon to gather org/network data.
3. Perform DNS enumeration (A, MX, NS, TXT, CNAME).

Tools — Browser, WHOIS lookup (built-in whois/Sysinternals WHOIS on Windows, or whois.domaintools.com), nslookup/dig.

Where to practice — All of this is passive (public-source) or light active querying — safe against any domain you're authorized to research (your college's own domain, or a well-known domain like google.com for a demo), but never run it as a prelude to actually attacking that target.

Step-by-step

1. Resolve the domain's IP:

nslookup www.google.com

2. Enumerate each DNS record type separately:

nslookup -type=A google.com

nslookup -type=MX google.com

nslookup -type=NS google.com

nslookup -type=TXT google.com

nslookup -type=CNAME www.google.com

Record	Reveals
A	IPv4 address of the host
MX	Mail servers handling the domain's email
NS	Authoritative name servers
TXT	SPF/DKIM/verification text
CNAME	Alias mapping one hostname to another

3. Reverse DNS (IP → hostname):

nslookup 142.251.152.119

4. WHOIS lookup — registrar, creation/expiry date, registrant org, name servers:

whois google.com

5. HTTP header inspection (server banner, security headers):

curl -I https://google.com

6. Search-engine recon ("Google dorking") — passive, no traffic sent to the target itself:

site:google.com filetype:pdf

site:google.com inurl:login

site:google.com intitle:"login"

7. Save findings for your report:

nslookup example.com > footprint_dns.txt

Likely viva Qs: Difference between passive and active footprinting? Which record type finds a domain's mail server? What's ARIN's role?

Topic 4 — Password Security, Dictionary Attacks, CrypTool

Aim

1. Study password entropy, hashing and salting, and why they matter for crackability.
2. Demonstrate a dictionary attack against a password you set yourself, to show why weak/common passwords fail fast.
3. Use CrypTool to visualize encryption vs. hashing algorithms.

Tools — Python 3, CrypTool (free, open-source — download CrypTool 1 or CT2 from cryptool.org), PowerShell Get-FileHash.

Step-by-step

1. **Password entropy** — a quick Python script to estimate strength (entropy ≈ length × log2(charset size)):

```
import math

pwd = input("Enter a test password: ")

charset = 0

if any(c.islower() for c in pwd): charset += 26

if any(c.isupper() for c in pwd): charset += 26

if any(c.isdigit() for c in pwd): charset += 10

if any(not c.isalnum() for c in pwd): charset += 32

entropy = len(pwd) * math.log2(charset) if charset else 0

print(f"Length: {len(pwd)}")

print(f"Estimated entropy: {entropy:.1f} bits")
```

2. **Unsalted vs salted hashing** — shows why salting defeats rainbow tables:

```
import os, hashlib

password = b"TestPass123"

print("Unsalted SHA-256 (user1):", hashlib.sha256(password).hexdigest())

print("Unsalted SHA-256 (user2):", hashlib.sha256(password).hexdigest())  # identical!

s1 = hashlib.scrypt(password, salt=os.urandom(16), n=16384, r=8, p=1, dklen=20).hex()

s2 = hashlib.scrypt(password, salt=os.urandom(16), n=16384, r=8, p=1, dklen=20).hex()
```

```
print("Salted scrypt (user1):", s1)

print("Salted scrypt (user2):", s2)  # different, even though password is the same
```

3. **Dictionary attack against your own hash** — create a short wordlist, hash a test password yourself, then show how fast a weak password is recovered:

```
import hashlib


# Step A: set a weak password and hash it (simulating a stored hash)

my_password = "password123"

target_hash = hashlib.sha256(my_password.encode()).hexdigest()


# Step B: a small sample wordlist (use a real wordlist file in the exam, one word per line)

wordlist = ["123456", "qwerty", "password123", "letmein", "admin"]


for word in wordlist:

   if hashlib.sha256(word.encode()).hexdigest() == target_hash:

      print(f"Password cracked: {word}")

      break

else:

   print("Not found in wordlist")
```

Run it once more after changing my_password to something long/random (e.g. Tr@il#92!qZx) — the same wordlist fails, demonstrating why entropy matters.

4. **File integrity hashing (PowerShell)** — shows the avalanche effect:

```
"Hello, Cryptography Lab!" | Set-Content message.txt

Get-FileHash .\message.txt -Algorithm SHA256

"Hello, Cryptography Lab!!" | Set-Content message.txt   # one char added

Get-FileHash .\message.txt -Algorithm SHA256           # completely different hash
```

5. **CrypTool** — open CrypTool → Encrypt/Decrypt menu → pick a cipher (e.g. AES, RSA) → type plaintext → generate a key → observe ciphertext. Then separately: Indiv. Procedures → Hash menu → pick SHA-256/MD5 → type text → observe digest, and change one character to show the avalanche effect visually (same idea as step 4, but with CrypTool's built-in visualizer instead of PowerShell).

Encryption vs hashing (viva table):

	Encryption	**Hashing**
Reversible?	Yes, with the key	No, one-way
Purpose	Confidentiality	Integrity / password storage
Examples	AES, RSA	SHA-256, bcrypt, Argon2

Likely viva Qs: Why is salting necessary even after hashing? Why shouldn't you use plain SHA-256/MD5 for password storage? What's the difference between a dictionary attack and brute force?

Topic 5 — Wireshark & Network Traffic Analysis

Aim

1. Capture and inspect network packets.
2. Identify common protocols such as DNS, ICMP, TCP and ARP.
3. Use display filters to find relevant traffic.

Tools: Wireshark, Windows PowerShell, optionally tshark.

Step-by-step

1. Check your network configuration

ipconfig /all

Identify your active network adapter and IPv4 address.

2. Generate sample network traffic

ping example.com

This generates ICMP traffic if the destination and network permit it.

You can also resolve a domain:

Resolve-DnsName example.com

This generates DNS activity, although the result may come from a cache.

3. Capture packets in Wireshark

1. Open Wireshark.
2. Select your active network interface, such as Wi-Fi or Ethernet.
3. Start capturing.
4. Run the PowerShell commands above.
5. Stop the capture.
6. Inspect the packets in the packet list and details panes.

4. Apply display filters

Enter these in Wireshark's display filter bar:

Filter	**Purpose**
arp	Displays ARP packets
dns	Displays DNS traffic
icmp	Displays ICMP packets, including ping
tcp	Displays TCP packets
ip.addr == 192.168.56.102	Displays packets involving the lab IP

Replace the example IP with your actual lab target.

5. Identify packet details

Select a packet and inspect:

- Source: The sender's address.
- Destination: The receiver's address.

- Protocol: Such as DNS, ICMP, TCP or ARP.
- Length: Packet size.
- Info: A brief description of the packet.

6. Save your capture

In Wireshark, select File → Save As and save the capture as:

lab-capture.pcapng

If tshark is installed, you can inspect a saved capture from PowerShell:

tshark -r "$HOME\lab-capture.pcapng" -Y "arp"

Likely viva questions

1. What is Wireshark? A network protocol analyzer used to capture and inspect packets.
2. What is packet sniffing? Capturing and examining network packets.
3. What is a display filter? A filter that limits which captured packets are shown.
4. What is the difference between capture and display filters? Capture filters control what is collected; display filters control what is shown after capture.
5. What does DNS do? It translates domain names into IP addresses and provides other domain-related records.
6. What is ARP? Address Resolution Protocol maps IPv4 addresses to link-layer addresses on a local network.

Topic 6 — Python Keylogger Practical

Aim

1. Understand how keyboard input can be captured by a program.
2. Demonstrate keyboard-input handling using Python.
3. Understand the security and privacy risks of keyloggers.

Tools: Windows PowerShell, Python, Notepad or another code editor.

Step-by-step

1. Check whether Python is installed

python --version

If that does not work, try:

py --version

2. Create the demonstration program

Run:

notepad "$HOME\keyboard_demo.py"

Paste this code into Notepad and save the file:

while True: text = input("Enter sample text (type exit to stop): ")   if text.lower() == "exit":   print("Demo stopped.")   break   print("Input received:", text)

3. Execute the program

python "$HOME\keyboard_demo.py"

If you use the Python launcher instead:

py "$HOME\keyboard_demo.py"

Enter some sample text. The program displays the text entered into its own prompt. Type exit to stop it.

4. Observe the output

Example:

Enter sample text (type exit to stop): Hello

Input received: Hello

Enter sample text (type exit to stop): exit

Demo stopped.

Observation: The program reads input deliberately provided through its own console. It is a safe input-handling demonstration, not a full system-wide keylogger.

5. Understand the security risk

A malicious keylogger may attempt to capture passwords, messages or other sensitive input. Defences include updated security software, downloading applications only from trusted sources, checking suspicious processes and avoiding untrusted scripts.

Likely viva questions

1. What is a keylogger? Software or hardware designed to record keyboard input.
2. What is the purpose of this practical? To understand keyboard-input handling and the security risks of input capture.
3. Which Python function reads console input? input().
4. Why is a keylogger dangerous? It can expose passwords, personal messages and other confidential information.
5. How can keyloggers be prevented? Use trusted software, updated security tools and good system-security practices.
6. Is the demonstration a full keylogger? No. It reads only text entered into its own console prompt.

Topic 7 — ARP Poisoning & Cain & Abel

Aim

1. Understand ARP and how local-network address resolution works.
2. Learn the concept of ARP poisoning and its possible effects.
3. Identify suspicious ARP changes and understand defensive measures.

Tools: Windows PowerShell, Wireshark, an isolated VirtualBox lab.

Where to practice: Inspect your own computer or an authorized isolated lab. This practical focuses on detection and defence, not poisoning devices or intercepting other users' traffic.

Step-by-step

1. Display the ARP cache

arp -a

This displays IPv4-to-MAC-address mappings known to your computer.

Look for the interface heading, Internet Address, Physical Address and Type columns.

2. Inspect neighbour entries

Get-NetNeighbor -AddressFamily IPv4

This displays IPv4 neighbour-cache entries and their states.

3. Check your network configuration

Get-NetIPConfiguration

Identify the active interface and default gateway.

4. Generate normal local-network traffic

ping <your-default-gateway-IP>

arp -a

Replace the placeholder with the default gateway address shown by Get-NetIPConfiguration.

Compare the ARP cache before and after the ping. The gateway's entry may appear or be refreshed.

5. Inspect ARP packets in Wireshark

1. Open Wireshark.
2. Select the active network interface.
3. Start capturing.
4. Use the display filter: arp
5. Generate ordinary traffic to your own gateway.
6. Stop the capture and inspect ARP requests and replies.

Look for unexpected changes in the mapping between an IP address and a MAC address. One unusual entry alone does not prove an attack; legitimate network changes can also alter mappings.

6. Understand ARP poisoning

ARP poisoning (also called ARP spoofing) occurs when forged ARP messages cause a device to associate an IP address, often the gateway's, with an incorrect MAC address. This can enable traffic redirection or disruption on a vulnerable local network.

Cain & Abel: This is a legacy Windows security tool historically associated with password recovery and network-security demonstrations. It is not a built-in PowerShell command, and availability or compatibility may vary. For the practical, explain its historical role and use Wireshark and PowerShell to demonstrate detection instead.

7. Defensive measures

- Use managed switches with Dynamic ARP Inspection where supported.
- Enable appropriate switch-port security.
- Monitor unexpected IP-to-MAC changes.
- Segment networks and secure access to network infrastructure.

- Use encrypted protocols such as HTTPS to reduce the impact of traffic interception.

Likely viva questions

1. What is ARP? A protocol used to resolve IPv4 addresses to MAC addresses on a local network.
2. What is ARP poisoning? Sending forged ARP messages to manipulate IP-to-MAC mappings.
3. What is a possible effect of ARP poisoning? Traffic redirection, interception or network disruption.
4. How can suspicious ARP activity be detected? By examining ARP packets and checking for unexpected IP-to-MAC changes.
5. What is Dynamic ARP Inspection? A managed-switch security feature that validates ARP messages against trusted binding information.
6. What is Cain & Abel? A legacy Windows security tool used historically for password recovery and network-security testing.